Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	Exp_1

AIM:	8 Puzzle
PROBLEM STATEMENT :	Start from given state and try to achieve the goal state in 8 puzzle problem
Theory:	Think of the 8-puzzle like rearranging heavy boxes in a tiny room with only one empty space: instead of calculating thousands of moves, humans naturally solve it by "chunking"—getting the top row of tiles in order first, mentally locking them in place, and only shuffling the remaining pieces. When we teach AI to solve it, we give it a similar sense of intuition called a heuristic (like the "Manhattan Distance"), where the computer simply asks, "If there were no other tiles in the way, how many steps would each piece have to walk to get to its final home?" and always chooses the move that brings the overall board closer to the perfect picture.

PROGRAM:	#include <iostream>
	#include <vector>
	#include <array>
	#include <iomanip>
	#include <random>
	
	using namespace std;
	
	const int N = 3;
	array<array<int, N>, N> goal;
	
	struct Node {
	  array<array<int, N>, N> mat;
	  int x, y;
	  int g, h, f;
	  char move;
	};

```
void printMat(const array<array<int, N>, N>& mat) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << setw(2) << mat[i][j] << " ";
        }
        cout << "\n";
    }
}

int calH(const array<array<int, N>, N>& mat) {
    int count = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (mat[i][j] != 0 && mat[i][j] != goal[i][j]) {
                count++;
            }
        }
    }
    return count;
}

Node createNode(const Node& parent, int newX, int newY, char move) {
    Node node;
    node.mat = parent.mat;
    swap(node.mat[newX][newY], node.mat[parent.x][parent.y]);

    node.x = newX;
    node.y = newY;
    node.g = parent.g + 1;
    node.h = calH(node.mat);
    node.f = node.g + node.h;
    node.move = move;

    return node;
}

bool isReverse(char prev, char curr) {
    if (prev == 'L' && curr == 'R') return true;
    if (prev == 'R' && curr == 'L') return true;
```

```
    if (prev == 'U' && curr == 'D') return true;
    if (prev == 'D' && curr == 'U') return true;
    return false;
}

bool isGoalReached(const array<array<int, N>, N>& mat) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (mat[i][j] != goal[i][j]) {
                return false;
            }
        }
    }
    return true;
}
void solve(const array<array<int, N>, N>& start, int x, int y) {
    Node current;
    current.mat = start;
    current.x = x;
    current.y = y;
    current.g = 0;
    current.h = calH(start);
    current.f = current.h;
    current.move = 'S';

    int step = 0;

    random_device rd;
    mt19937 gen(rd());

    while (true) {
        cout << "\n====================== STEP " << step << "
======================\n";
        cout << "\nCurrent Matrix : \n";
        printMat(current.mat);
        cout << "g = " << current.g << ", h = " << current.h << ", f = " << current.f << "
\n";

        if (isGoalReached(current.mat)) {
            cout << "\nGoal Reached!\n";
```

```
    return;
}

vector<Node> children;
int dx[] = {0, 0, -1, 1};
int dy[] = {-1, 1, 0, 0};
char moves[] = {'L', 'R', 'U', 'D'};

cout << "\nPossible Moves : \n";
for (int i = 0; i < 4; i++) {
    int nx = current.x + dx[i];
    int ny = current.y + dy[i];

    if (nx >= 0 && nx < N && ny >= 0 && ny < N) {
        if (isReverse(current.move, moves[i])) {
            continue;
        }

        Node child = createNode(current, nx, ny, moves[i]);
        children.push_back(child);

        cout << "\n Move to " << moves[i] << " : \n";
        printMat(child.mat);
        cout << " g = " << child.g << ", h = " << child.h << ", f = " << child.f << " \n";
    }
}

if (children.empty()) {
    cout << "\nNo valid moves available.\n";
    return;
}

int minF = children[0].f;
for (const auto& child : children) {
    if (child.f < minF) {
        minF = child.f;
    }
}

vector<int> candidates;
```

```
        for (size_t i = 0; i < children.size(); i++) {
            if (children[i].f == minF) {
                candidates.push_back(i);
            }
        }

        uniform_int_distribution<> dist(0, candidates.size() - 1);
        int chosenIndex = candidates[dist(gen)];

        cout << "\n Chosen Move (min f) : \n";
        printMat(children[chosenIndex].mat);
        cout << "g = " << children[chosenIndex].g << ", h = " << children[chosenIndex].h
<< ", f = " << children[chosenIndex].f << " \n";

        current = children[chosenIndex];
        step++;

        if (step > 1000) {
            cout << "\nMaximum steps exceeded. The algorithm is stuck in a local loop.\n";
            return;
        }
    }
}

int main() {
    array<array<int, N>, N> start;
    int x = -1, y = -1;

    cout << "Enter the Initial Matrix (9 numbers, use 0 for empty space): \n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cin >> start[i][j];
            if (start[i][j] == 0) {
                x = i;
                y = j;
            }
        }
    }
    cout << "Enter the Goal Matrix (9 numbers): \n";
    for (int i = 0; i < N; i++) {
```

```
        for (int j = 0; j < N; j++) {
            cin >> goal[i][j];
        }
    }

    if (x == -1 || y == -1) {
        cout << "Error: No empty space (0) found in the initial matrix.\n";
        return 1;
    }

    solve(start, x, y);
    return 0;
}
```

Output:

```
students@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ g++ puzzle_8.cpp -o my_program
students@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ ./my_program
Enter the Initial Matrix (9 numbers, use 0 for empty space):
1 2 3
0 4 6
7 5 8
Enter the Goal Matrix (9 numbers):
1 2 3
4 5 6
7 8 0

====================== STEP 0 ======================

Current Matrix :
 1  2  3
 0  4  6
 7  5  8
g = 0, h = 3, f = 3

Possible Moves :

 Move to R :
 1  2  3
 4  0  6
 7  5  8
 g = 1, h = 2, f = 3

 Move to U :
 0  2  3
 1  4  6
 7  5  8
 g = 1, h = 4, f = 5

 Move to D :
 1  2  3
 7  4  6
 0  5  8
 g = 1, h = 4, f = 5
```

```
 Chosen Move (min f) :
 1  2  3
 4  0  6
 7  5  8
g = 1, h = 2, f = 3

==================== STEP 1 ====================

Current Matrix :
 1  2  3
 4  0  6
 7  5  8
g = 1, h = 2, f = 3

Possible Moves :

 Move to R :
 1  2  3
 4  6  0
 7  5  8
 g = 2, h = 3, f = 5

 Move to U :
 1  0  3
 4  2  6
 7  5  8
 g = 2, h = 3, f = 5

 Move to D :
 1  2  3
 4  5  6
 7  0  8
 g = 2, h = 1, f = 3
```

```
 Chosen Move (min f) :
 1  2  3
 4  5  6
 7  0  8
g = 2, h = 1, f = 3

==================== STEP 2 ====================

Current Matrix :
 1  2  3
 4  5  6
 7  0  8
g = 2, h = 1, f = 3

Possible Moves :

 Move to L :
 1  2  3
 4  5  6
 0  7  8
 g = 3, h = 2, f = 5

 Move to R :
 1  2  3
 4  5  6
 7  8  0
 g = 3, h = 0, f = 3

 Chosen Move (min f) :
 1  2  3
 4  5  6
 7  8  0
g = 3, h = 0, f = 3
```

```
==================== STEP 3 ====================

Current Matrix :
 1  2  3
 4  5  6
 7  8  0
g = 3, h = 0, f = 3

Goal Reached!
```

CONCLUSION:	Ultimately, implementing a solution for the 8-puzzle problem is a classic demonstration of how human intuition can be translated into algorithmic efficiency. By pairing a systematic search method such as the A algorithm with a smart heuristic like Manhattan Distance, the code avoids blindly wandering through hundreds of thousands of possible board combinations. Instead, it systematically evaluates and prioritizes the most promising moves, proving how giving a computer a simple rule of thumb can transform a computationally overwhelming puzzle into an elegantly and swiftly solvable task.